

From Coin Flips to Language Models

Part 1: How do we represent information and uncertainty mathematically?

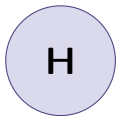
Sarvin Motamen

University of Virginia

September 1, 2026

- 1 From coin flips to language models
- 2 Finite-context models
- 3 Learning and generation
- 4 Context size and parameter sharing
- 5 Summary

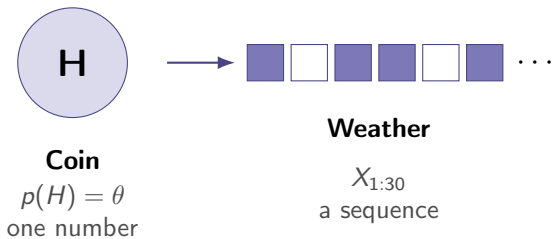
From coin flips to sequences



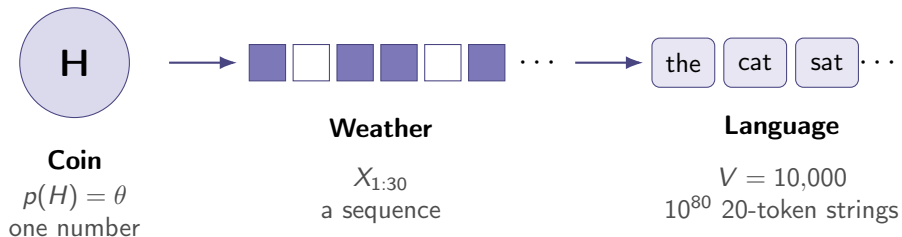
Coin

$p(H) = \theta$
one number

From coin flips to sequences



From coin flips to sequences



All are joint distributions. Sequences need more structure.

Why the full joint fails

A full joint table assigns a probability to every complete weather record.

Three days

0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

$2^3 = 8$ sequences

$2^3 - 1 = 7$ free probabilities

Why the full joint fails

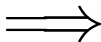
A full joint table assigns a probability to every complete weather record.

Three days

0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

$2^3 = 8$ sequences

$2^3 - 1 = 7$ free probabilities



Thirty days



$2^{30} - 1 = 1,073,741,823$
free probabilities

Why the full joint fails

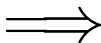
A full joint table assigns a probability to every complete weather record.

Three days

0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

$2^3 = 8$ sequences

$2^3 - 1 = 7$ free probabilities



Thirty days

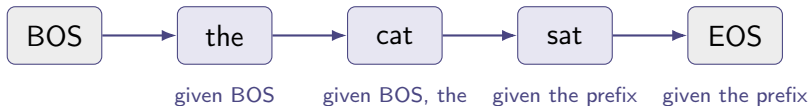


$2^{30} - 1 = 1,073,741,823$
free probabilities

A realistic dataset contains little or no evidence about most 30-day sequences.

The chain rule gives next-token predictions

$$p(w_{1:T}) = \prod_{t=1}^T p(w_t \mid w_{1:t-1})$$

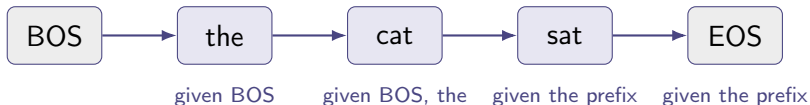


Exact factorization

A joint distribution becomes a left-to-right sequence of next-token distributions.

The chain rule gives next-token predictions

$$p(w_{1:T}) = \prod_{t=1}^T p(w_t \mid w_{1:t-1})$$



Exact factorization

A joint distribution becomes a left-to-right sequence of next-token distributions.

The chain rule keeps the whole prefix. A later assumption will shorten it.

Next-token probabilities define the joint distribution

For a sequence of tokens $w_{1:T} = (w_1, \dots, w_T)$, the chain rule gives

$$p(w_{1:T}) = \prod_{t=1}^T p(w_t \mid w_{1:t-1})$$

- Each factor is a categorical distribution over the next token.
- Multiplying the selected probabilities scores the whole sequence.
- Sampling from the conditionals, left to right, generates a sequence.

Next-token probabilities define the joint distribution

For a sequence of tokens $w_{1:T} = (w_1, \dots, w_T)$, the chain rule gives

$$p(w_{1:T}) = \prod_{t=1}^T p(w_t \mid w_{1:t-1})$$

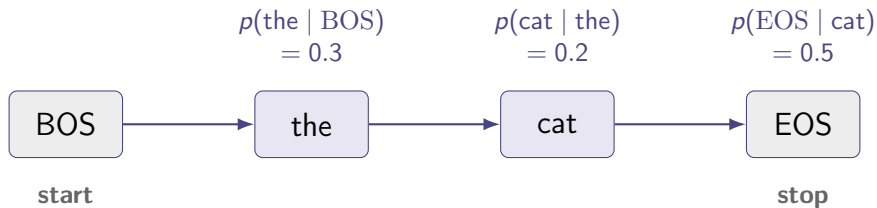
- Each factor is a categorical distribution over the next token.
- Multiplying the selected probabilities scores the whole sequence.
- Sampling from the conditionals, left to right, generates a sequence.

► Warning: factorization $\neq$ simplification

Every factor may still use the entire prefix, so the parameter count is unchanged.

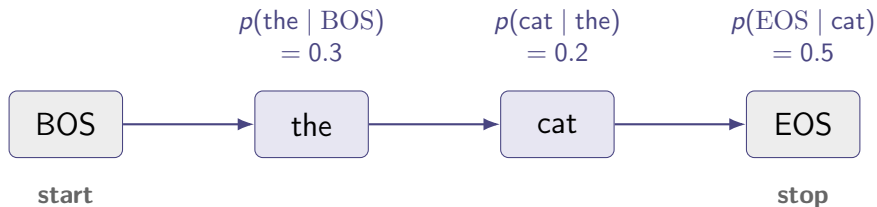
Boundary tokens handle beginnings and endings

One-token contexts: a complete generated sequence



Boundary tokens handle beginnings and endings

One-token contexts: a complete generated sequence



Probability of this completed sequence

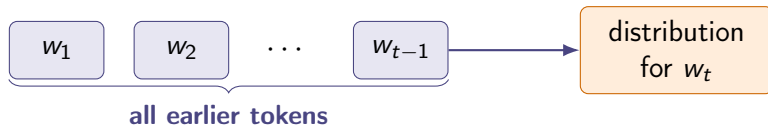
$$p(\text{the, cat, EOS} \mid \text{BOS}) = 0.3 \times 0.2 \times 0.5 = \boxed{0.03}.$$

Outline

- 1 From coin flips to language models
- 2 Finite-context models**
- 3 Learning and generation
- 4 Context size and parameter sharing
- 5 Summary

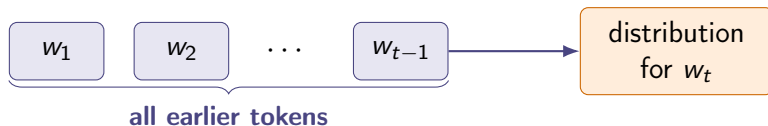
Exact prediction uses the entire prefix

$$p(w_t \mid \underbrace{w_1, w_2, \dots, w_{t-1}}_{\text{entire prefix}})$$



Exact prediction uses the entire prefix

$$p(w_t \mid \underbrace{w_1, w_2, \dots, w_{t-1}}_{\text{entire prefix}})$$



Different prefix $\implies$ different table row.

An order- k model keeps the last k tokens

$$p(w_t \mid w_{1:t-1}) \underset{\text{modeling assumption}}{\approx} p_k(w_t \mid w_{t-k:t-1}).$$

Order	Context	Next-token distribution
0	no preceding token	$p_0(w_t)$
1	previous token	$p_1(w_t \mid w_{t-1})$
2	previous two tokens	$p_2(w_t \mid w_{t-2}, w_{t-1})$

An order- k model keeps the last k tokens

$$p(w_t \mid w_{1:t-1}) \underset{\text{modeling assumption}}{\approx} p_k(w_t \mid w_{t-k:t-1}).$$

Order	Context	Next-token distribution
0	no preceding token	$p_0(w_t)$
1	previous token	$p_1(w_t \mid w_{t-1})$
2	previous two tokens	$p_2(w_t \mid w_{t-2}, w_{t-1})$

We will use the same context-to-next-token distribution at every position in the sequence.

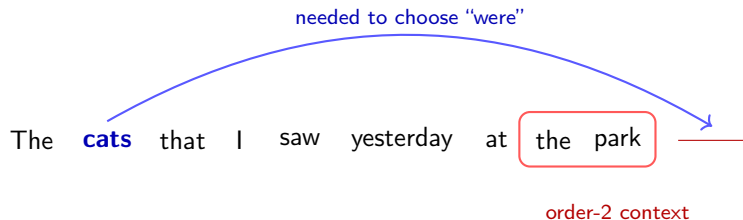
A lookup table has $V^k(V - 1)$ free parameters

$$\underbrace{V^k}_{\text{context rows}} \times \underbrace{(V - 1)}_{\text{free per row}} = V^k(V - 1)$$

Order	Context rows	Free / row	Free parameters
0	1	9,999	9,999
1	10^4	9,999	$\approx 10^8$
2	10^8	9,999	$\approx 10^{12}$

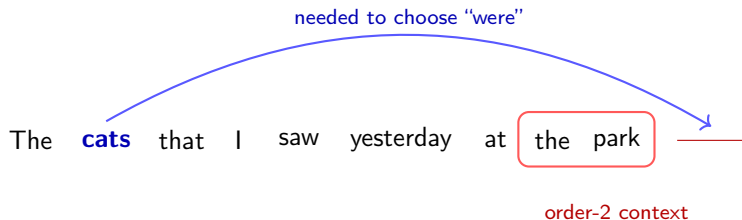
For $V = 10,000$. One probability in each row is fixed because the row sums to one.

A short context can miss information that matters



An order-2 model sees only "the park." It cannot use the distant plural subject "cats."

A short context can miss information that matters



An order-2 model sees only "the park." It cannot use the distant plural subject "cats."

The trade-off begins here

Larger k can capture longer dependencies, but creates many more contexts to estimate.

Outline

- 1 From coin flips to language models
- 2 Finite-context models
- 3 Learning and generation**
- 4 Context size and parameter sharing
- 5 Summary

Counts estimate one distribution per context

For a context c , count how often each token follows it:

$$N(c, w) = \#\{\text{times } w \text{ follows } c\}, \quad N(c) = \sum_u N(c, u).$$

Normalize the row:

$$\hat{p}_k(w \mid c) = \frac{N(c, w)}{N(c)}$$

Counts estimate one distribution per context

For a context c , count how often each token follows it:

$$N(c, w) = \#\{\text{times } w \text{ follows } c\}, \quad N(c) = \sum_u N(c, u).$$

Normalize the row:

$$\hat{p}_k(w \mid c) = \frac{N(c, w)}{N(c)}$$

Example. If (the, cat) is followed once by “sat” and once by “slept,” then

$$\hat{p}_2(\text{sat} \mid \text{the, cat}) = \hat{p}_2(\text{slept} \mid \text{the, cat}) = \frac{1}{2}.$$

Counts estimate one distribution per context

For a context c , count how often each token follows it:

$$N(c, w) = \#\{\text{times } w \text{ follows } c\}, \quad N(c) = \sum_u N(c, u).$$

Normalize the row:

$$\hat{p}_k(w \mid c) = \frac{N(c, w)}{N(c)}$$

Example. If (the, cat) is followed once by “sat” and once by “slept,” then

$$\hat{p}_2(\text{sat} \mid \text{the, cat}) = \hat{p}_2(\text{slept} \mid \text{the, cat}) = \frac{1}{2}.$$

Observed context

$$N(c) > 0$$

Unseen continuation

$$\begin{aligned} N(c, w) &= 0 \Rightarrow \\ \hat{p}_k(w \mid c) &= 0 \end{aligned}$$

One small corpus gives an order-2 table

Corpus: the cat sat; the cat slept; the dog sat; a cat sat.

Pad each sentence with two BOS tokens and one EOS.

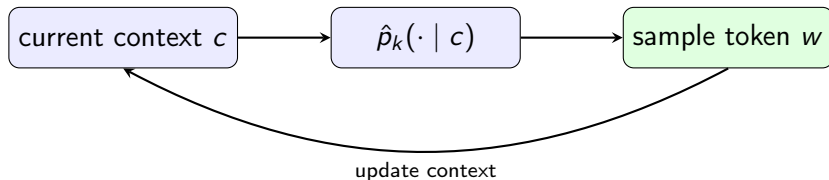
Context c	Nonzero counts $N(c, w)$	Estimated $\hat{p}_2(w c)$
(BOS, BOS)	the: 3, a: 1	the: 3/4, a: 1/4
(BOS, the)	cat: 2, dog: 1	cat: 2/3, dog: 1/3
(BOS, a)	cat: 1	cat: 1
(the, cat)	sat: 1, slept: 1	sat: 1/2, slept: 1/2
(the, dog)	sat: 1	sat: 1
(a, cat)	sat: 1	sat: 1
(cat, sat)	EOS: 2	EOS: 1
(cat, slept)	EOS: 1	EOS: 1
(dog, sat)	EOS: 1	EOS: 1

Only nonzero entries are shown; every row sums to one.

Generation is predict, sample, append, repeat

For an order- k model:

- 1 Start with k copies of BOS.
- 2 Sample w from $\hat{p}_k(\cdot | c)$.
- 3 If $w = \text{EOS}$, stop; otherwise append w .
- 4 Keep the newest k tokens as the next context and repeat.

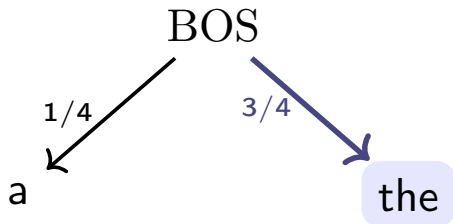


A sampled path also has a probability

Relevant rows of the order-2 table

Context c	$\hat{p}_2(w \mid c)$
(BOS, BOS)	the: 3/4, a: 1/4
(BOS, the)	cat: 2/3, dog: 1/3
(the, cat)	sat: 1/2, slept: 1/2
(cat, slept)	EOS: 1

One sampled path

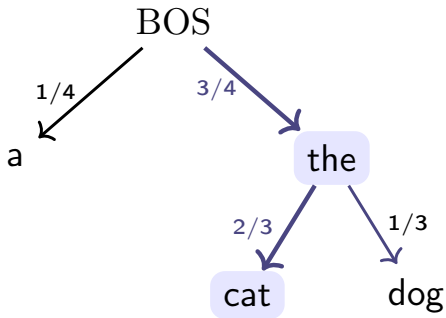


A sampled path also has a probability

Relevant rows of the order-2 table

Context c	$\hat{p}_2(w c)$
(BOS, BOS)	the: 3/4, a: 1/4
(BOS, the)	cat: 2/3, dog: 1/3
(the, cat)	sat: 1/2, slept: 1/2
(cat, slept)	EOS: 1

One sampled path

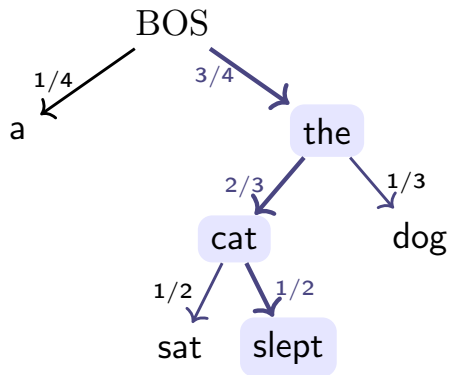


A sampled path also has a probability

Relevant rows of the order-2 table

Context c	$\hat{p}_2(w c)$
(BOS, BOS)	the: 3/4, a: 1/4
(BOS, the)	cat: 2/3, dog: 1/3
(the, cat)	sat: 1/2, slept: 1/2
(cat, slept)	EOS: 1

One sampled path

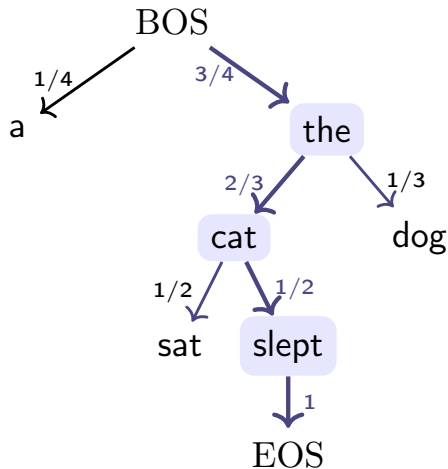


A sampled path also has a probability

Relevant rows of the order-2 table

Context c	$\hat{p}_2(w c)$
(BOS, BOS)	the: 3/4, a: 1/4
(BOS, the)	cat: 2/3, dog: 1/3
(the, cat)	sat: 1/2, slept: 1/2
(cat, slept)	EOS: 1

One sampled path



A sampled path also has a probability

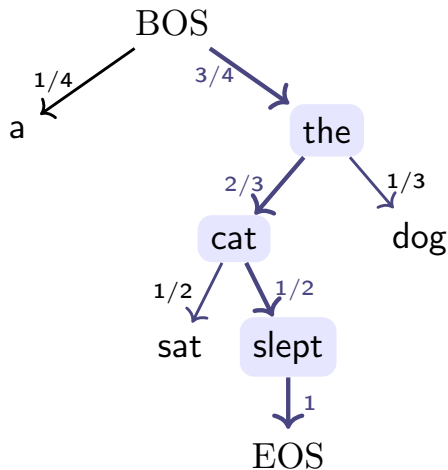
Relevant rows of the order-2 table

Context c	$\hat{p}_2(w c)$
(BOS, BOS)	the: 3/4, a: 1/4
(BOS, the)	cat: 2/3, dog: 1/3
(the, cat)	sat: 1/2, slept: 1/2
(cat, slept)	EOS: 1

Multiply the highlighted edges:

$$\hat{p}_2(\text{the cat slept, EOS}) = \frac{3}{4} \times \frac{2}{3} \times \frac{1}{2} \times 1 = \boxed{\frac{1}{4}}.$$

One sampled path



Sampling is not the same as taking the maximum

Suppose a row assigns

$$\hat{p}(\text{sat} \mid c) = 0.6, \quad \hat{p}(\text{slept} \mid c) = 0.4.$$

Sampling

- “sat” is chosen about 60% of the time.
- “slept” is chosen about 40% of the time.
- Repeated runs can differ.

Always take the maximum

- “sat” is always chosen.
- The path is deterministic, apart from ties.
- This is not sampling from the model.

Outline

- 1 From coin flips to language models
- 2 Finite-context models
- 3 Learning and generation
- 4 Context size and parameter sharing**
- 5 Summary

More context trades data for detail

In the same four-sentence corpus:

Order 1: pool evidence across contexts

$$\hat{p}_1(\text{sat} \mid \text{cat}) = \frac{2}{3}, \quad \hat{p}_1(\text{slept} \mid \text{cat}) = \frac{1}{3}.$$

It can generate the new combination “a cat slept.”

More context trades data for detail

In the same four-sentence corpus:

Order 1: pool evidence across contexts

$$\hat{p}_1(\text{sat} \mid \text{cat}) = \frac{2}{3}, \quad \hat{p}_1(\text{slept} \mid \text{cat}) = \frac{1}{3}.$$

It can generate the new combination “a cat slept.”

Order 2: keep contexts separate

$$\hat{p}_2(\text{slept} \mid \text{a, cat}) = 0.$$

It has only seen “a cat sat,” so it cannot make that combination.

More context trades data for detail

In the same four-sentence corpus:

Order 1: pool evidence across contexts

$$\hat{p}_1(\text{sat} \mid \text{cat}) = \frac{2}{3}, \quad \hat{p}_1(\text{slept} \mid \text{cat}) = \frac{1}{3}.$$

It can generate the new combination “a cat slept.”

Order 2: keep contexts separate

$$\hat{p}_2(\text{slept} \mid \text{a, cat}) = 0.$$

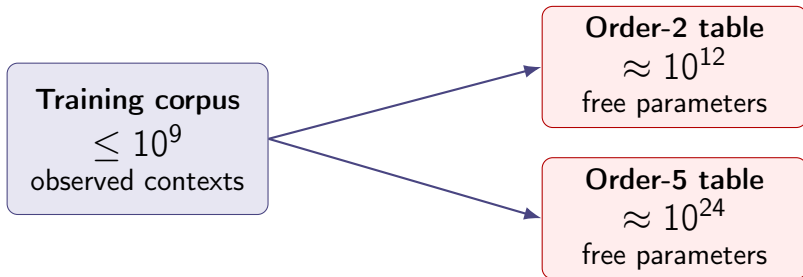
It has only seen “a cat sat,” so it cannot make that combination.

	Benefit	Cost with fixed data
Small k	more observations per context	ignores useful history
Large k	represents finer distinctions	sparse or unseen contexts

Neither behavior is automatically better: the useful context must be supported by enough observations.

Tables run out of data

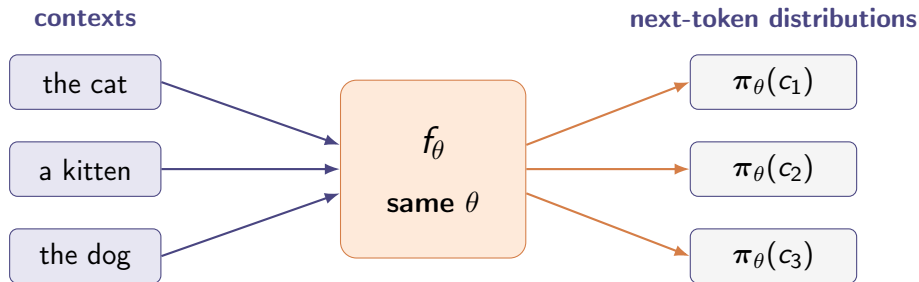
Non-parametric lookup table: one separate row for every context



Most rows are empty

A row can be learned only from occurrences of its own context.

One shared function answers every context



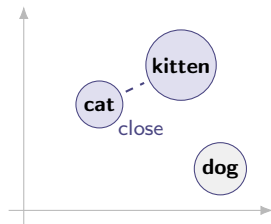
Parametric model

One fixed parameter set is shared across contexts.

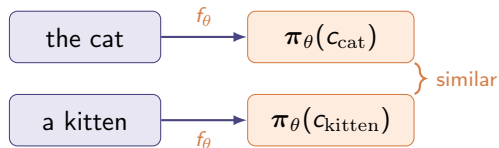
Learning in one context can affect predictions in another.

Why shared functions can generalize

Learned embedding space



Similar inputs, similar next-token distributions



A neural network is the shared function that maps context vectors to probabilities.

Longer context without a separate table row

Count tables allocate separate parameters to each context. A neural model shares them.

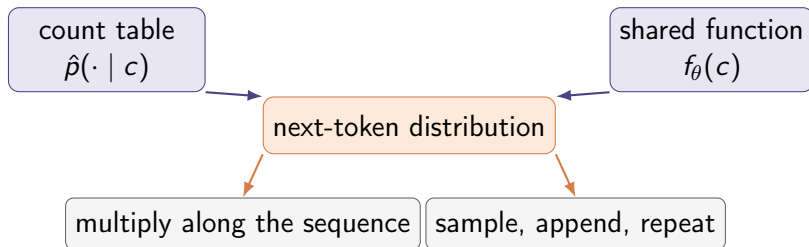
Model	Context	Free parameters
Order-2 count table	2 tokens	$\approx 10^{12}$
Order-5 count table	5 tokens	$\approx 10^{24}$
Neural language model	thousands of tokens	10^9 to 10^{12}

Count-table values assume a vocabulary of $V = 10,000$.

The scaling advantage

A neural language model can condition on hundreds or thousands of times more tokens than a count table, because its parameters are shared across contexts.

What stays the same



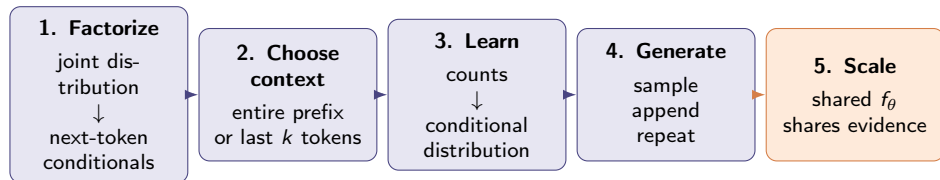
$$p_{\theta}(w_{1:T}) = \prod_{t=1}^T p_{\theta}(w_t | w_{1:t-1})$$

Next: choose θ by making the observed data probable.

Outline

- 1 From coin flips to language models
- 2 Finite-context models
- 3 Learning and generation
- 4 Context size and parameter sharing
- 5 Summary**

What should stay with you



The core trade-off

More context gives more detail, but less data per context.